МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Введение в искусственный интеллект и логическое программирование»

РЕКУРСИВНЫЕ СТРУКТУРЫ ДАННЫХ В ПРОЛОГЕ. ДЕРЕВЬЯ. ГРАФЫ

НОВОСИБИРСК 2026

1. Цель работы

Приобретение навыков создания баз знаний с использованием фактов и правил, реализация диалоговых программ для обработки данных и выполнения запросов к базе знаний.

2. Задание

1. Исследовать работу логической программы, задающей отношения над объектами рекурсивных типов:
   а) записать предложенную программу на Турбо-Прологе;
   б) выполнить программу в пошаговом режиме и построить дерево вывода при ее работе;
   в) модифицировать программу, вставляя в предложения программы предикат отсечения (!);
   г) выполнить модифицированную программу в пошаговом режиме и построить дерево вывода;
   д) сравнить деревья вывода в том и другом случае и определить «цвет» отсечения.

Задание для исследования:

Сравнение деревьев с точностью до изоморфизма

2. Разработать и отладить программу решения индивидуального задания.

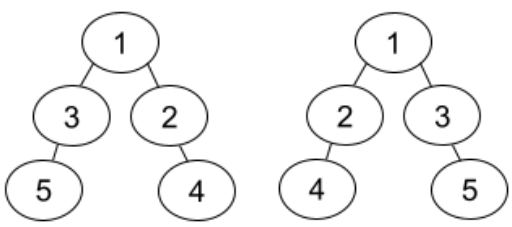
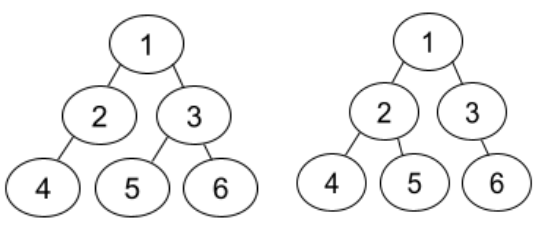
Индивидуальное задание:

Напишите программу, которая преобразует логическую формулу F1 в логическую формулу F2, используя следующие аксиомы:

true V F=true и false & F=false, где F – логическая формула.

3. Описание разработанной программы

3.1. Сравнение деревьев с точностью до изоморфизма

Изоморфные деревья	Неизоморфные деревья
	

3.1.1. Сравнение деревьев без предиката отсечения

1) Структура базы знаний:
    - tree = empty; tr(integer, tree, tree) — рекурсивная структура бинарного дерева
2) Основные предикаты:
    - isotree(tree, tree) — проверяет изоморфизм двух бинарных деревьев
3) Принцип работы: программа сравнивает два дерева рекурсивно. Деревья изоморфны, если оба пусты, либо если корни равны, а поддеревья изоморфны в прямом порядке (левое с левым, правое с правым) или в обратном порядке (левое с правым, правое с левым). Без отсечения программа находит все возможные пути доказательства изоморфизма, для симметричных листьев дает несколько решений (4 решения для изоморфных деревьев, 0 для неизоморфных).

3.1.2. Сравнение деревьев с предикатом отсечения

1) Структура базы знаний:
    - tree = empty; tr(integer, tree, tree) — рекурсивная структура бинарного дерева
2) Основные предикаты:
    - isotree(tree, tree) — проверяет изоморфизм двух бинарных деревьев
3) Принцип работы: программа сравнивает два дерева рекурсивно аналогично версии без отсечения. Отсечение ! добавлено в конец каждого правила после успешной проверки. Это исключает поиск альтернативных путей доказательства для симметричных листьев. Для изоморфных деревьев программа находит 1 решение вместо 4, для неизоморфных — 0 решений. Отсечение является зеленым, так как не меняет декларативный смысл программы.

3.2. Преобразование логических формул

1) Структура базы знаний:
    - formula = true; false; v(symbol); d(formula, formula); c(formula, formula) — рекурсивная структура логической формулы (константы true/false, переменная, дизъюнкция d(F1,F2), конъюнкция c(F1,F2))
2) Основные предикаты:
    - simplify(formula, formula) — упрощает логическую формулу, применяя аксиомы true | F = true и false & F = false

- write_f(formula) — выводит формулу в читаемом виде

3) Принцип работы: программа рекурсивно обходит формулу. Если встречается дизъюнкция с true, вся подформула заменяется на true по аксиоме true | F = true. Если встречается конъюнкция с false, вся подформула заменяется на false по аксиоме false & F = false. В остальных случаях программа рекурсивно спускается в подформулы. Отсечение используется для предотвращения лишнего поиска после успешного применения аксиомы. Аксиомы применяются за один проход по формуле.

4. Примеры запросов к базе знаний и ее ответов

4.1.1. Сравнение деревьев без предиката отсечения

Изоморфные:

```
Izomorfnye trees
Resheniya:
 - izomorfny
Tree1=tr(1,tr(2,tr(4,empty,empty),empty),tr(3,empty,tr(5,empty,empty))), Tree2=tr(1,tr(3,tr(5,empty,empty),empty),tr(2,empty,tr(4,empty,empty)))
 - izomorfny
Tree1=tr(1,tr(2,tr(4,empty,empty),empty),tr(3,empty,tr(5,empty,empty))), Tree2=tr(1,tr(3,tr(5,empty,empty),empty),tr(2,empty,tr(4,empty,empty)))
 - izomorfny
Tree1=tr(1,tr(2,tr(4,empty,empty),empty),tr(3,empty,tr(5,empty,empty))), Tree2=tr(1,tr(3,tr(5,empty,empty),empty),tr(2,empty,tr(4,empty,empty)))
 - izomorfny
Tree1=tr(1,tr(2,tr(4,empty,empty),empty),tr(3,empty,tr(5,empty,empty))), Tree2=tr(1,tr(3,tr(5,empty,empty),empty),tr(2,empty,tr(4,empty,empty)))
4 Solutions
```

Неизоморфные:

```
Neizomorfnye trees
Resheniya:
No Solution
```

4.1.2. Сравнение деревьев с предикатом отсечения

Изоморфные:

```
Izomorfnye trees
Resheniya:
 - izomorfny
Tree1=tr(1,tr(2,tr(4,empty,empty),empty),tr(3,empty,tr(5,empty,empty))), Tree2=tr(1,tr(3,tr(5,empty,empty),empty),tr(2,empty,tr(4,empty,empty)))
1 Solution
```

Неизоморфные:

```
Neizomorfnye trees
Resheniya:
No Solution
```

4.2. Преобразование логических формул

```
#1. F1 = true | (a & b)
    F2 = true

#2. F1 = false & (x | y)
    F2 = false

#3. F1 = p | (false & q)
    F2 = (p | false)

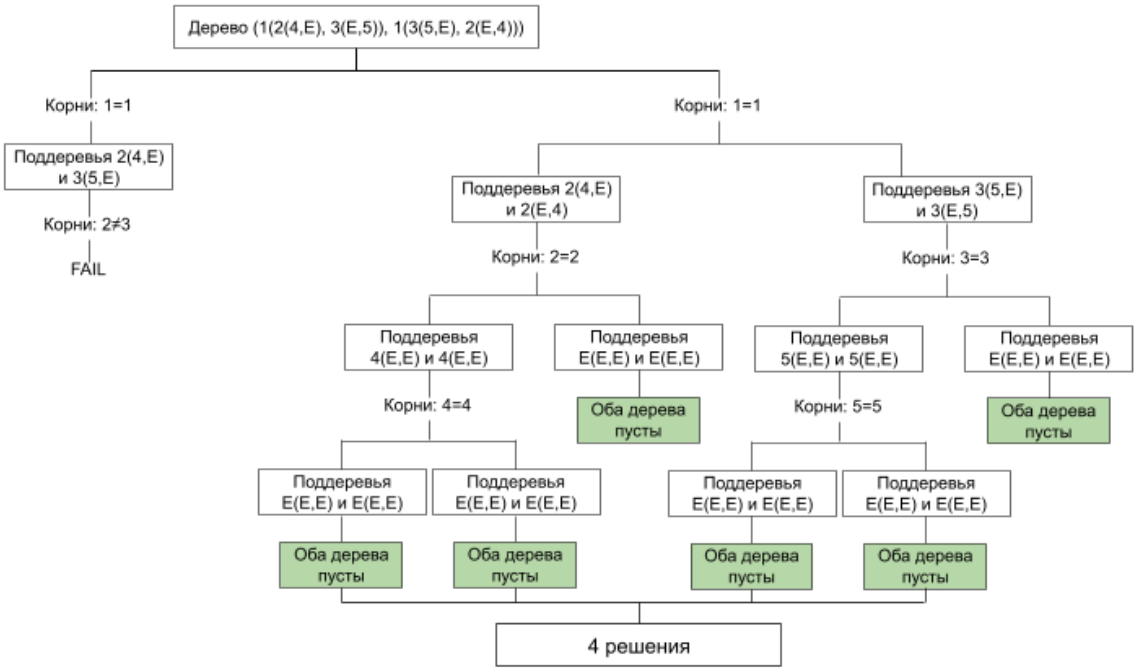
#4. F1 = (true | x) & y
    F2 = (true & y)

#5. F1 = (false & a) | b
    F2 = (false | b)

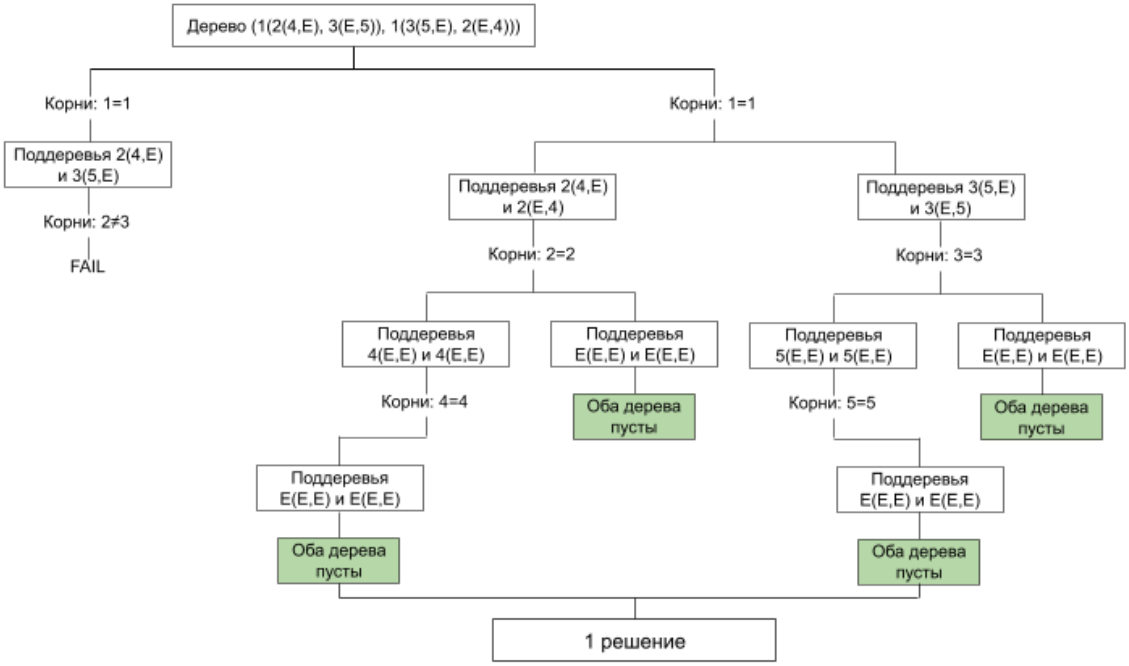
F2=true, F4=false, F6=d(v('p'),false), F8=c(true,v('y')), F10=d(false,v('b'))
1 Solution
```

5. Дерево поиска

5.1.1. Сравнение деревьев без предиката отсечения



5.1.2. Сравнение деревьев с предикатом отсечения



6. Листинг разработанной базы знаний на языке Пролог

6.1.1. Сравнение деревьев без предиката отсечения

```
DOMAINS
    tree = empty; tr(integer, tree, tree)

PREDICATES
    nondeterm isotree(tree, tree)

CLAUSES
    % Oba dereva pusty
    isotree(empty, empty).

    % Korni ravny, podderevya izomorfny v pryamom poryadke
    isotree(tr(X, L1, R1), tr(X, L2, R2)) :-
        isotree(L1, L2),
        isotree(R1, R2).

    % Korni ravny, podderevya izomorfny v obratnom poryadke
    isotree(tr(X, L1, R1), tr(X, L2, R2)) :-
        isotree(L1, R2),
        isotree(R1, L2).

GOAL
    write("Neizomorfnye trees"), nl,

    % Iz
    % Tree1 = tr(1, tr(2, tr(4, empty, empty), empty), tr(3, empty, tr(5, empty,
empty))),
    % Tree2 = tr(1, tr(3, tr(5, empty, empty), empty), tr(2, empty, tr(4, empty,
empty))),

    % Ne iz
    Tree1 = tr(1, tr(2, tr(4, empty, empty), tr(5, empty, empty)), tr(3, empty, tr(6,
empty, empty))),
    Tree2 = tr(1, tr(2, tr(4, empty, empty), empty), tr(3, tr(5, empty, empty), tr(6,
empty, empty))),
```

```
    write("Resheniya:"), nl,
    isotree(Tree1, Tree2), write("  - izomorfny"), nl.
```

6.1.2. Сравнение деревьев с предикатом отсечения

```
DOMAINS
    tree = empty; tr(integer, tree, tree)

PREDICATES
    nondeterm isotree(tree, tree)

CLAUSES
    % Oba dereva pusty
    isotree(empty, empty) :- !.

    % Korni ravny, podderevya izomorfny v pryamom poryadke
    isotree(tr(X, L1, R1), tr(X, L2, R2)) :-
        isotree(L1, L2),
        isotree(R1, R2),
        !.

    % Korni ravny, podderevya izomorfny v obratnom poryadke
    isotree(tr(X, L1, R1), tr(X, L2, R2)) :-
        isotree(L1, R2),
        isotree(R1, L2),
        !.

GOAL
    write("Izomorfnye trees"), nl,

    % Iz
    Tree1 = tr(1, tr(2, tr(4, empty, empty), empty), tr(3, empty, tr(5, empty,
empty))),
    Tree2 = tr(1, tr(3, tr(5, empty, empty), empty), tr(2, empty, tr(4, empty,
empty))),

    % Ne iz
    % Tree1 = tr(1, tr(2, tr(4, empty, empty), tr(5, empty, empty)), tr(3, empty,
tr(6, empty, empty))),
    % Tree2 = tr(1, tr(2, tr(4, empty, empty), empty), tr(3, tr(5, empty, empty),
tr(6, empty, empty))),

    write("Resheniya:"), nl,
    isotree(Tree1, Tree2), write("  - izomorfny"), nl.
```

6.2. Преобразование логических формул

```
DOMAINS
    formula = true; false; v(symbol); d(formula, formula); c(formula, formula)

PREDICATES
    simplify(formula, formula)
    write_f(formula)

CLAUSES
    % True | F = true
    simplify(d(true, _), true) :- !.
    simplify(d(_, true), true) :- !.

    % False & F = false
    simplify(c(false, _), false) :- !.
    simplify(c(_, false), false) :- !.

    % Rekursivnyi spusk dlya dizyunktsii
    simplify(d(F1, F2), d(S1, S2)) :-
        !,
        simplify(F1, S1),
```

```
        simplify(F2, S2).

    % Rekursivnyi spusk dlya konyunktsii
    simplify(c(F1, F2), c(S1, S2)) :-
        !,
        simplify(F1, S1),
        simplify(F2, S2).

    % Bazovyi sluchai
    simplify(F, F).

    write_f(true) :- write("true").
    write_f(false) :- write("false").
    write_f(v(X)) :- write(X).
    write_f(d(A, B)) :-
        write("("), write_f(A), write(" | "), write_f(B), write(")").
    write_f(c(A, B)) :-
        write("("), write_f(A), write(" & "), write_f(B), write(")").

GOAL
    write("#1. F1 = true | (a & b)"), nl,
    simplify(d(true, c(v(a), v(b))), F2),
    write("      F2 = "), write_f(F2), nl, nl,

    write("#2. F1 = false & (x | y)"), nl,
    simplify(c(false, d(v(x), v(y))), F4),
    write("      F2 = "), write_f(F4), nl, nl,

    write("#3. F1 = p | (false & q)"), nl,
    simplify(d(v(p), c(false, v(q))), F6),
    write("      F2 = "), write_f(F6), nl, nl,

    write("#4. F1 = (true | x) & y"), nl,
    simplify(c(d(true, v(x)), v(y)), F8),
    write("      F2 = "), write_f(F8), nl, nl,

    write("#5. F1 = (false & a) | b"), nl,
    simplify(d(c(false, v(a)), v(b)), F10),
    write("      F2 = "), write_f(F10), nl, nl.
```